

Guia de Repertório: POO & Escopos

Conceitos Fundamentais de Programação Orientada a Objetos, Funções e Escopos de Variáveis

1. Orientação a Objetos: Classes e Objetos

Em programação e na Orientação a Objetos (POO), uma **classe** é um molde ou blueprint usado para criar objetos. Ela define quais características (atributos) e comportamentos (métodos) os objetos terão.

Analogia: A Planta de uma Casa

Classe: É a planta arquitetônica. Não é uma casa de verdade, mas especifica as medidas, cômodos e regras.

Objeto (Instância): É a casa física construída usando a planta. A partir da mesma planta, você constrói várias casas com cores e acabamentos distintos.

Estrutura Básica da Classe

- **Atributos (Dados/Características):** Variáveis que guardam o estado do objeto.
- **Métodos (Ações/Comportamentos):** Funções que executam ações ou alteram os dados.

```
# Definição da Classe
class Carro:
    def __init__(self, marca, cor):
        self.marca = marca # Atributo
        self.cor = cor     # Atributo

    def acelerar(self):     # Método
        print(f"O {self.marca} está acelerando!")

# Criando Objetos (Instanciação)
meu_carro = Carro("Toyota", "Preto")
outro_carro = Carro("Ford", "Azul")

meu_carro.acelerar() # Saída: O Toyota está acelerando!
```

2. Funções e Parâmetros

Funções e parâmetros caminham juntos para tornar o código reutilizável, modular e organizado.

1. O que é uma Função?

Bloco de código projetado para executar uma tarefa específica. Em vez de repetir instruções, você chama a função sempre que precisar.

Analogia: Pense em um **liquidificador** que executa a tarefa de triturar ao apertar um botão.

2. O que são Parâmetros?

Entradas que a função precisa para trabalhar. Variáveis reservadas para receber valores no momento da execução.

Analogia: São as **frutas e o leite** colocados dentro do liquidificador.

Parâmetro vs. Argumento

Parâmetro: Nome da variável definido na criação da função (ex: nome).

Argumento: Valor real passado ao chamar a função (ex: "Ana").

3. O Comando Return

O comando `return` envia o resultado de uma função de volta para o ponto onde ela foi chamada.

Sem return

A função apenas executa uma ação (ex: imprime na tela). O valor processado não fica guardado e tentar armazená-lo resulta em `None`.

Com return

A função faz a conta ou processamento e devolve o valor final, permitindo que ele seja salvo em variáveis ou usado em novos cálculos.

```
def calcular_desconto(preco, percentual):
    valor_desconto = preco * (percentual / 100)
    preco_final = preco - valor_desconto
    return preco_final # Devolve o valor calculado

# Guardando o resultado retornado
valor_para_pagar = calcular_desconto(200, 15) # 15% de desconto em R$ 200
print(f"O valor a pagar é R$ {valor_para_pagar:.2f}")

# Reutilizando o valor retornado em outra operação
valor_com_frete = valor_para_pagar + 10
print(f"Com frete, fica: R$ {valor_com_frete:.2f}")
```

Regras Importantes do Return:

1. **Encerra a execução:** No instante em que o `return` é atingido, a função para imediatamente.
2. **Reutilização:** Permite salvar em banco de dados, usar em novas contas ou enviar como parâmetro.

4. Escopo Local vs. Escopo Global

A diferença entre os dois resume-se a **onde** uma variável pode ser acessada ou modificada no código.

Analogia do Apagador de Lousa

Variável Global: Quadro branco no corredor da escola. Visível para todos e qualquer um pode alterar.

Variável Local: Bloco de anotações na mochila. Acessível apenas enquanto você o usa, invisível para quem está fora.

Característica	Escopo Global	Escopo Local
Declaração	Fora de todas as funções	Dentro de uma função específica
Acesso	Em qualquer parte do programa	Apenas dentro da função onde nasceu
Tempo de vida	Enquanto o programa estiver rodando	Criada na chamada, destruída ao terminar
Isolamento	Risco alto de alterações indesejadas	Isolado e seguro contra alterações externas

```
mensagem_global = "Eu sou global!"

def minha_funcao():
    mensagem_local = "Eu sou local!"
    print(mensagem_global) # OK! Lê a variável global.
```

5. Modificando Variáveis Globais

Por padrão, atribuir um valor a uma variável dentro de uma função cria uma nova variável local. Para alterar a global, é preciso declará-la explicitamente.

Como fazer em Python (instrução `global`)

```
contador = 0

def incrementar():
    global contador # Avisa que usaremos a variável global 'contador'
    contador += 1   # Altera diretamente a variável global

incrementar()
incrementar()
print(contador) # Saída: 2
```

O que acontece sem a palavra global? (Shadowing Accidental)

O Python cria uma variável local de mesmo nome, sem alterar a variável global externa.

```
contador = 0

def tentar_incrementar():
    contador = 10 # Cria uma variável LOCAL 'contador'
    print(f"Dentro: {contador}") # Saída: 10

tentar_incrementar()
print(f"Fora: {contador}")      # Saída: 0 (A global permaneceu intocada)
```

6. Sombreamento de Variável (Variable Shadowing)

Ocorre quando uma variável local tem o **mesmo nome** de uma variável global. A variável local oculta a global dentro daquele escopo, recebendo prioridade absoluta.

Analogia dos Apelidos

O vizinho de bairro se chama Marcos (global). Na rua, ao chamar "Marcos!", refere-se ao vizinho. Se você contratar um funcionário chamado Marcos (local) para trabalhar na sua casa, dentro de casa "Marcos" sempre será o funcionário. O vizinho continua lá fora intocado, mas seu nome foi "sombreado" dentro de casa.

Erro Comum: `UnboundLocalError`

Acontece em Python ao tentar ler uma variável global e depois atribuir um valor local a ela dentro da mesma função. O interpretador marca a variável como local para toda a função desde o início, gerando erro de leitura prévia.

```
x = 10

def testar():
    # print(x) <-- Lança UnboundLocalError!
    x = 20      # A atribuição faz o Python tratar 'x' como local em TODA a função.

testar()
```